NOTIFYHUB

Centralized Digital Notice & Communication Platform

“Never miss a campus update”

Software Requirements, Architecture & Development Documentation (Next.js Edition)

Prepared by: Koppula Yashwanth Reddy

Document Version: 2.0

Date: 3 August 2026

Table of Contents

1. Abstract

1.1 Justification for the Proposed Digital Solution

Educational institutions rely on timely, accurate communication to function well. Notices about examinations, fee deadlines, placement drives, faculty meetings, and emergency alerts all need to reach the right audience quickly and reliably. In most colleges and universities today, this is still handled through physical notice boards, WhatsApp groups, email chains, or a patchwork of departmental circulars. As an institution grows in size and complexity, this informal approach breaks down: notices get missed, duplicated, or reach students after the relevant deadline has passed.

NotifyHub is proposed as a single, centralized, role-aware communication system that gives every stakeholder — from the Admin down to prospective students browsing the public site — a clear, structured channel to publish, discover, and respond to information. The justification rests on three pillars: (a) reducing the time-to-communicate for urgent notices from hours/days to minutes, (b) providing accountability and traceability of who published what and when, and (c) giving students a direct, monitored channel to raise queries to the Admin with an enforced response-time target (under 6 hours).

1.2 Analysis of Existing Notice Board Systems

1.2.1 Traditional (Physical) Notice Boards

- No real-time delivery; a notice is only visible to someone who physically walks past the board.
- No record of who saw the notice, whether it was read, or when it was taken down.
- No way to target a notice at a specific department or year group without printing multiple copies.
- No search or history — once removed, a past notice is effectively lost.
- Physically constrained — limited board space forces older notices to be removed prematurely.

1.2.2 Ad-hoc Digital Alternatives (Email, WhatsApp, Generic Portals)

- Email is easily missed amid unrelated traffic and has no urgency differentiation.
- WhatsApp groups are informal, hard to moderate, not role-based, and not auditable.
- Generic portals rarely distinguish an announcement, an event, and an urgent alert, and offer no live countdown or urgency signal.
- None enforce a response-time commitment for prospective-student or general-public queries.

1.3 How NotifyHub Addresses These Shortcomings

- A role-based hierarchy (Admin → Principal → Dean → HoD → Faculty), each with a dedicated dashboard, ensures every notice has a clear, authorized publisher.
- Announcements, Events, and Alerts are modeled as distinct content types, with Alerts visually flagged in red and a live countdown for time-bound items.
- A built-in search and filter engine lets any visitor locate a notice quickly instead of scrolling a physical board or a chat history.
- A single, unauthenticated Query form on the Home page lets prospective/current students reach the Admin directly — no login required — with responses tracked against a 6-hour target.
- Secure, hashed authentication (bcrypt) and strict role-based authorization ensure only the correct staff role can create or edit content, and that only the Admin can add/delete a user account or delete a notice once published.

- Built entirely on Next.js, the platform ships as a single full-stack project — pages, dashboards, and APIs all deploy together on Vercel with no separate backend service to manage.

2. Stakeholders & Role-Based Access Control (RBAC)

NotifyHub's authorization model sits on top of a clear hierarchy. Admin is the single highest authority in the system — superior to Principal — and is the only role with total control: adding or deleting any staff account, and deleting any notice once it has gone live. Principal, Dean, HoD, and Faculty each get their own dedicated dashboard scoped to their level, but none of them can delete content or manage accounts. Students have no login at all; they interact with the platform purely as anonymous visitors to the public pages, with a single unauthenticated channel — the Home page query form — to reach the Admin.

Role	Scope	Key Permissions
Admin	Entire institution — top authority, superior to Principal	Total control over the platform: add and delete any staff account (Principal, Dean, HoD, Faculty) through the Account Manager; the only role that can delete a notice once it has been published; publish institution-wide content; receives and answers every student query submitted through the Home page; full audit log and analytics access.
Principal	Institution-wide, subordinate to Admin	Publish and edit institution-wide announcements, events, and alerts from a dedicated Principal dashboard; oversee Deans' and HoDs' communications; view aggregated cross-faculty analytics. Cannot delete notices and cannot manage user accounts — both are Admin-only.
Dean	Faculty-level	Publish/edit notices and events for their faculty from a dedicated Dean dashboard; oversee HoDs and Faculty within the faculty; view faculty-level analytics. No delete access, no account management.
HoD	Department-level	Publish/edit department notices from a dedicated HoD dashboard; review Faculty-authored notices within the department. No delete access, no account management.
Faculty	Course/section-level	Post/edit notices and events for their courses from a dedicated Faculty dashboard. No delete access, no account management.
Student (public visitor)	No login — anonymous	Browses Home, Announcements, Events, and Alerts; submits a query only through the Home page's Student Query Form (Name, Roll No, Department, Query). No account, no dashboard, and the form is never shown on any staff dashboard.

2.1 Admin — Total Control

Admin sits above Principal in the hierarchy and is the only role that can: (a) add a new staff account for any role, (b) delete any staff account, (c) delete a published notice of any type, and (d) view and respond to every student query. This concentration of destructive/administrative power in a single role keeps accountability unambiguous and avoids the accidental or unilateral removal of content or accounts by lower-privileged staff.

2.2 Student Access — No Login

Students are not modeled as an authenticated role in this version of NotifyHub. They browse the public pages freely and, if they need to reach the institution, use the Student Query Form on the Home page — not a Contact page, and not any staff dashboard — by supplying their Name, Roll Number, Department, and Query. This keeps the barrier to asking a question at zero while keeping every staff-facing page free of a form that has no relevance to staff.

3. Requirements Overview

3.1 Functional Requirements

- The system shall support role-based login for Admin, Principal, Dean, HoD, and Faculty; students shall not require any login.
- The system shall let Admin, Principal, Dean, HoD, and Faculty create and edit Announcements, Events, and Alerts within their scope, each from their own dedicated dashboard.
- The system shall restrict the deletion of a published notice exclusively to Admin.
- The system shall restrict adding and deleting staff accounts exclusively to Admin, via an Account Manager.
- The system shall visually distinguish Alerts using a red color scheme and support a live countdown for time-bound alerts.
- The system shall provide keyword search and category/department/date filters across all notice types on the public pages.
- The system shall present a Student Query Form — capturing Name, Roll No, Department, and Query — only on the Home (“/”) route, and never on any Admin/Principal/Dean/HoD/Faculty dashboard.
- The system shall not require authentication to submit a student query, and shall route every submitted query to the Admin's Query inbox with elapsed-time tracking against a 6-hour response target.
- The Home page shall display a live count of total notices and upcoming events, plus a short explainer on why NotifyHub exists.
- The system shall expose a webhook mechanism so new announcements/alerts can trigger downstream notifications.
- All passwords shall be stored using one-way bcrypt hashing; plaintext passwords shall never be persisted or logged.

3.2 Non-Functional Requirements

- Performance: Home page and notice lists should render within 2 seconds, aided by Next.js server-side rendering and caching.
- Scalability: The system should comfortably support a multi-faculty university without redesign.
- Availability: Target uptime of 99.5%, consistent with Vercel's managed Next.js hosting.
- Security: Role-based authorization on every route (enforced both in middleware and inside each API handler); delete and user-management actions gated to Admin alone; hashed credentials; strict validation/rate-limiting on the unauthenticated query endpoint.
- Usability: Mobile-first, responsive UI built with Tailwind CSS — the Student Query Form in particular must work smoothly on a phone, since that is the primary access point for students.

- Auditability: Every content create/update/delete action and every account-management action is logged with actor, timestamp, and target.
- Maintainability: A single Next.js codebase (pages, dashboards, and API/Route Handlers together) to avoid the coordination overhead of separate frontend/backend deployments.

4. Technology Stack & System Architecture

4.1 Technology Stack

Layer	Technology	Purpose
Framework	Next.js (App Router), React, Tailwind CSS	Unified full-stack framework — pages, dashboards, and API layer all live in one project.
Backend / API	Next.js Route Handlers (app/api/**/route.ts)	Replaces the standalone Express server; runs as Vercel serverless/edge functions.
Security	bcrypt, signed session cookies / JWT	Password hashing and staff authentication.
Database	PostgreSQL (Neon, serverless) via Prisma or Drizzle ORM	Relational storage for users, announcements, events, alerts, and queries.
Route Protection	Next.js Middleware (middleware.ts)	Edge-level role checks on /admin, /principal, /dean, /hod, /faculty before a request reaches the page or API.
Deployment / Hosting	Vercel — native Next.js deployment	One project, zero manual build scripts; frontend and API deploy together automatically.
Version Control	Git & GitHub	Source control; PR-based review; Vercel preview deployments per pull request.
Real-time layer	Webhooks (+ optional Route Handler-based SSE)	Push new announcements/alerts to connected clients and external integrations.

4.2 High-Level Architecture

NotifyHub is a single Next.js project. Public pages (Home, Announcements, Events, Alerts) are rendered largely as Server Components, fetching directly from Postgres for fast, cache-friendly reads. Each staff role gets its own protected route segment (/admin, /principal, /dean, /hod, /faculty) guarded by middleware.ts and re-verified inside every API handler. All writes — create, edit, delete, account management — go through Route Handlers under app/api/**, which run as Vercel serverless/edge functions alongside the rendered pages.

- Presentation Tier: Next.js App Router — public pages as Server Components, interactive pieces (Student Query Form, dashboard forms) as Client Components, styled with Tailwind CSS.
- Application Tier: Next.js Route Handlers implementing the REST-style API in Section 6.3, with middleware.ts performing the first layer of role checks.
- Data Tier: Neon serverless PostgreSQL via Prisma/Drizzle, storing normalized tables for users/roles, announcements, events, alerts, and queries.

- Integration Layer: A webhook dispatcher fires on notice creation/update so downstream systems (push notifications, email digests, Slack/Teams) can subscribe without coupling to the core app.

4.3 Deployment on Vercel

Because the entire application — pages and API — is one Next.js project, deployment no longer needs the manual 'build the frontend, copy it into a backend public folder' workaround required by a separate React + Express setup. Vercel has native, zero-configuration support for Next.js:

- Connect the GitHub repository to a new Vercel project — Vercel auto-detects Next.js and sets the correct build/output settings; no vercel.json or custom builds/routes config is needed.
- Every push to main triggers a production deployment; every pull request gets its own preview URL, useful for staff review before a feature goes live.
- Add environment variables in Project Settings: DATABASE_URL (Neon pooled connection string), the session/JWT secret, and any webhook URLs — never commit these to the repo.
- Use Neon's pooled connection string (the '-pooler' host) or the @neondatabase/serverless driver, since Route Handlers run as serverless/edge functions and can spin up many concurrent database connections.
- Next.js Middleware runs at the edge, so role-based route protection for /admin, /principal, /dean, /hod, and /faculty is enforced globally with minimal latency, before a protected page or API route is ever reached.

4.4 Core Database Entities

Table	Key Columns	Notes
users	id, name, email, password_hash, role, department_id, faculty_id, created_at	role enum: admin, principal, dean, hod, faculty. Only admin may INSERT or DELETE a row.
announcements	id, title, body, category, department_id, is_urgent, author_id, published_at, deleted_at	Only an admin-authenticated request may set deleted_at.
events	id, title, description, event_date, location, department_id, author_id, created_at, deleted_at	Powers the 'Upcoming Events' widget on the Home page.
alerts	id, title, body, expires_at, department_id, author_id, created_at, deleted_at	Drives the red-flagged Alerts page and its live countdown.
queries	id, student_name, roll_no, department, query_text, status, responded_at, created_at	No student_id / user reference — queries are unauthenticated submissions captured with the fields supplied on the Home page form.

5. Pages & Dashboards

NotifyHub now separates the site into public pages (open to everyone, no login) and role-specific dashboards (staff-only, protected by middleware).

5.1 Home Page (/ — Public)

- Live count of total notices and upcoming events.
- Short 'Why NotifyHub' explainer and the tagline "Never miss a campus update."

- Banner surfacing the most urgent active Alert.
- The Student Query Form — Name, Roll No, Department, Query — rendered only here. No authentication required to submit; the submission is a public POST to /api/queries.

5.2 Announcements / Events / Alerts Pages (Public, Read-Only)

- Chronological, filterable, searchable lists — rendered as Server Components for fast first load.
- No create/edit/delete controls appear here — all content management happens inside the role dashboards below.
- The Alerts page uses the red color scheme and a live countdown for time-bound alerts.

5.3 Admin Dashboard (/admin)

- Overview: institution-wide stats and recent activity.
- Account Manager (/admin/users): add a new staff account for any role, delete any staff account.
- Notice Management (/admin/notices): view, edit, or delete any Announcement, Event, or Alert across every department — the only dashboard with a delete control.
- Query Inbox (/admin/queries): view every student-submitted query and respond to it; the only place student queries are visible and answered.

5.4 Principal Dashboard (/principal)

- Publish and edit institution-wide announcements, events, and alerts.
- View aggregated analytics across all faculties/departments.
- No delete control, no Account Manager — both are Admin-only.

5.5 Dean Dashboard (/dean)

- Publish and edit notices/events for their own faculty.
- Oversee HoDs and Faculty within that faculty; view faculty-level analytics.
- No delete control, no Account Manager.

5.6 HoD Dashboard (/hod)

- Publish and edit department-level notices.
- Review Faculty-authored notices within the department.
- No delete control, no Account Manager.

5.7 Faculty Dashboard (/faculty)

- Post and edit notices/events relevant to their own courses/sections.
- No delete control, no Account Manager.

5.8 Staff Login (/login)

- Single login form for Admin, Principal, Dean, HoD, and Faculty; on success, the user is redirected to their role's dashboard root.
- There is no student login — students never see or use this page.

6. Low-Level Architecture

This section captures the concrete, implementation-level view of the Next.js build: the App Router folder structure, the middleware-based route protection, and the complete Route Handler table.

6.1 Project Folder Structure (App Router)

```
notifyhub/
├── app/
│   ├── page.tsx                    # Home "/" — public, includes StudentQueryForm
│   ├── announcements/page.tsx      # public, read-only
│   ├── events/page.tsx             # public, read-only
│   ├── alerts/page.tsx             # public, read-only
│   ├── login/page.tsx              # staff login only
│   ├── admin/
│   │   ├── page.tsx                # Admin dashboard overview
│   │   ├── users/page.tsx          # Account Manager — add/delete users
│   │   ├── notices/page.tsx        # manage & delete any notice
│   │   └── queries/page.tsx        # respond to student queries
│   ├── principal/page.tsx
│   ├── dean/page.tsx
│   ├── hod/page.tsx
│   ├── faculty/page.tsx
│   └── api/
│       ├── auth/login/route.ts
│       ├── auth/logout/route.ts
│       ├── users/route.ts              # GET, POST — admin only
│       ├── users/[id]/route.ts         # PUT, DELETE — admin only
│       ├── announcements/route.ts
│       ├── announcements/[id]/route.ts
│       ├── events/route.ts
│       ├── events/[id]/route.ts
│       ├── alerts/route.ts
│       ├── alerts/[id]/route.ts
│       ├── queries/route.ts            # POST public, GET admin
│       └── queries/[id]/respond/route.ts
├── components/
│   ├── StudentQueryForm.tsx        # rendered ONLY inside app/page.tsx
│   ├── NoticeCard.tsx
│   └── dashboards/                 # AdminSidebar, PrincipalPanel, DeanPanel, HodPanel,
FacultyPanel
├── lib/
│   ├── db.ts                       # Neon client / Prisma / Drizzle
│   ├── auth.ts                     # session/JWT helpers, bcrypt
│   └── rbac.ts                     # role-guard helpers used inside Route Handlers
├── middleware.ts                   # edge-level role protection
├── prisma/schema.prisma            # or drizzle schema
└── package.json
```

6.2 Route Protection Chain

- 1. middleware.ts intercepts every request to /admin/*, /principal/*, /dean/*, /hod/*, /faculty/*.
- 2. It reads the session cookie / JWT, decodes the role, and matches it against the required role for that path prefix (e.g., /admin/* requires role === 'admin').
- 3. On failure, it redirects to /login; on success, the request proceeds to the page or Route Handler.
- 4. Every Route Handler that mutates data re-checks the role itself (never trusts the middleware redirect alone), since middleware protects pages but API routes can in principle be called directly.
- 5. Route Handlers for DELETE (any notice) and all of /api/users hard-code an admin-only check with no other role ever added to that list.

- 6. /api/queries (POST) is intentionally left open — no session required — but is protected with input validation and rate limiting since it is the one public write endpoint in the system.

6.3 Complete Route Table

Method	Path	Role(s) Allowed	Description
POST	/api/auth/login	Public (staff only)	Authenticate staff credentials (bcrypt compare) and issue a session/JWT.
POST	/api/auth/logout	Authenticated	Clear the current session.
GET	/api/users	admin	List all staff accounts (Account Manager).
POST	/api/users	admin	Create a new staff account for any role.
DELETE	/api/users/:id	admin	Delete a staff account.
PUT	/api/users/:id	admin	Update a staff account's role/details.
GET	/api/announcements	Public	List/search/filter announcements.
POST	/api/announcements	admin, principal, dean, hod, faculty	Create an announcement.
PUT	/api/announcements/:id	author's role scope, admin	Edit an announcement.
DELETE	/api/announcements/:id	admin only	Delete a published announcement.
GET	/api/events	Public	List/search upcoming events.
POST	/api/events	admin, principal, dean, hod, faculty	Create an event.
PUT	/api/events/:id	author's role scope, admin	Edit an event.
DELETE	/api/events/:id	admin only	Delete a published event.
GET	/api/alerts	Public	List active alerts.
POST	/api/alerts	admin, principal, dean, hod, faculty	Create an alert.
PUT	/api/alerts/:id	author's role scope, admin	Edit an alert.
DELETE	/api/alerts/:id	admin only	Delete a published alert.
POST	/api/queries	Public, no auth	Student submits a query: name, roll_no, department, query_text.
GET	/api/queries	admin	List all student queries.
PUT	/api/queries/:id/respond	admin	Respond to a query; updates status and responded_at.

7. Phase-Wise Development Plan

Eight phases, from discovery through post-launch support, each defined by what is actually built during that phase.

Phase	What to Develop	Primary Deliverables
Phase 1 — Discovery & Requirements	Finalize the Admin-above-Principal hierarchy and the Admin-only delete/account-management rule; confirm the Student Query Form (Name, Roll No, Department, Query) lives only on Home; lock the page/dashboard scope; define functional/non-functional requirements.	Finalized requirements and role matrix.
Phase 2 — Design	Design the ERD & Postgres schema (users, announcements, events, alerts, queries); design the full Route Handler API contract (Section 6.3); wireframe the Home page, the three public notice pages, and each of the five dashboards; define Tailwind design tokens (including the Alert red); design the session/JWT auth flow and the middleware.ts role-protection rules.	DB schema, API contract, wireframes, security design.
Phase 3 — Full-Stack Foundation (Next.js)	Scaffold the App Router project per Section 6.1; implement bcrypt-based staff auth and session/JWT issuance; write middleware.ts; implement the Route Handlers for users (admin-only add/delete), announcements, events, alerts, and queries; implement the rbac.ts guard helpers reused across every handler.	Working API layer, testable via the Next.js dev server.
Phase 4 — Public Pages & Dashboards	Build Home (with the embedded Student Query Form), and the read-only Announcements/Events/Alerts pages as Server Components; build the Admin dashboard (Users, Notices, Queries); build the Principal, Dean, HoD, and Faculty dashboards with their own scoped create/edit features and no delete or user-management controls.	Fully functional site running against the local/staging database.
Phase 5 — Real-Time & Integration	Implement the webhook dispatcher fired on announcement/alert publish events; add the live notice/event counters on Home; implement the live countdown on the Alerts page; wire optional downstream integrations (email digest, Slack/Teams relay).	Live webhook endpoint; working real-time counters and countdown.
Phase 6 — Testing	Write Jest unit tests for the RBAC/service logic; write Route Handler integration tests against a Neon test branch, including explicit checks that only admin can reach /api/users and any DELETE route; test that the Student Query Form appears only on "/" and never in a dashboard; verify the public /api/queries endpoint is rate-limited; run UAT with Admin, Principal, Dean, HoD, and Faculty stakeholders.	Test reports and signed-off UAT.
Phase 7 — Deployment & Handover	Connect the GitHub repository to a new Vercel project (auto-detected as Next.js, no manual build scripts needed); configure environment variables (Neon pooled DATABASE_URL, session/JWT secret, webhook URLs); confirm middleware.ts is active in production; set up uptime/error monitoring; distribute the Admin handbook and role-specific quick-start guides for Principal, Dean, HoD, and Faculty.	Live production system on Vercel, monitoring in place, handbooks distributed.
Phase 8 — Post-Launch Support	Monitor error rates, webhook delivery, and query-submission volume/spam; run a bug-fix cycle from early staff feedback; review usage analytics (query SLA compliance, notice engagement); groom the backlog for v2 features (e.g., optional student accounts, push notifications).	Support log and v2 backlog.

8. Code-Level Optimization Strategies

Updated for the Next.js full-stack architecture, the Admin-above-Principal role model, and the unauthenticated public query channel.

8.1 Frontend (Next.js + Tailwind)

- Render Home, Announcements, Events, and Alerts as Server Components that query Postgres directly — avoids an extra client-side fetch round trip for read-only public content.
- Use Incremental Static Regeneration (a short revalidate window) on the public list pages so they stay fast under load while remaining reasonably fresh.
- Keep the Student Query Form, and every dashboard form, as a Client Component — the rest of Home stays server-rendered.
- Split each dashboard (/admin, /principal, /dean, /hod, /faculty) into its own route segment so a Faculty user's bundle never includes Admin's Account Manager code.
- Debounce the search input on the public notice pages before firing a request.
- Co-locate a small design-token layer (CSS variables for the Alert red) so the urgency color stays consistent across the Alerts page and the Home banner.

8.2 Backend (Next.js Route Handlers)

- Keep handlers thin: route → validate (Zod) → service/lib function → data-access layer, so business logic is testable independent of the Next.js request/response objects.
- Centralize role checks in lib/rbac.ts and call the same guard from every handler; treat DELETE routes and every /api/users route as their own hard-coded admin-only permission class.
- Use parameterized queries via Prisma/Drizzle to eliminate SQL-injection risk.
- Paginate every list endpoint by default.
- Rate-limit and strictly validate POST /api/queries — it is the only endpoint in the system that accepts writes with no authentication, so it is the most likely target for spam or abuse.
- Emit the webhook asynchronously (fire-and-retry) so a slow downstream subscriber never blocks the API response to the publishing staff member.

8.3 Route Protection (Middleware)

- Keep middleware.ts's role-to-path mapping in one small config object so adding a new protected segment is a one-line change, not a scattered set of conditionals.
- Never treat a middleware pass as sufficient authorization inside a Route Handler — re-verify the role server-side on every mutating request, since Route Handlers can be invoked directly.
- Fail closed: any request whose role can't be resolved (missing/invalid session) is redirected to /login rather than allowed through by default.

8.4 Database (Neon PostgreSQL)

- Index foreign keys used in scoping (department_id, faculty_id) and any column used in filters (is_urgent, category, published_at).
- Use a full-text search index (tsvector) for notice search rather than naive LIKE '%term%' queries.
- Enforce the role enum and query status enum with CHECK constraints, not just application-level validation.

- Use Neon's serverless branching to give each pull request its own isolated database branch for preview deployments.

8.5 Security

- Hash every staff password with bcrypt at an appropriate cost factor; never log or transmit plaintext passwords.
- Sign session tokens with a short expiry; store them httpOnly and secure.
- Sanitize all user-generated content, including the unauthenticated query_text field, to prevent stored XSS.
- Scope each staff JWT/session to their department/faculty so the backend rejects out-of-scope requests even if the UI is bypassed.
- Guard every DELETE route and every /api/users route with an explicit, non-configurable admin-only check in addition to generic auth.
- Log every Account Manager action and every delete action to an append-only audit table.

8.6 Testing & CI/CD

- Unit-test the rbac.ts guards and SLA-calculation logic independent of HTTP.
- Integration-test Route Handlers against a disposable Neon test branch, including an explicit test that every non-admin role receives a 403 on /api/users and every DELETE route.
- Add a Playwright test asserting the Student Query Form renders on "/" but is absent from /admin, /principal, /dean, /hod, and /faculty.
- Wire GitHub Actions to run lint → unit tests → integration tests → next build on every pull request, blocking merge on failure, before Vercel promotes the deployment.

9. Testing Strategy

9.1 Test Levels

Level	Focus	Tools/Approach
Unit	RBAC guards, SLA calculation, validation rules	Jest
Integration	Route Handlers against a real (test-branch) PostgreSQL database	Jest/Supertest-style + Neon test branch
End-to-End	Full flows: staff login → publish notice → public search → student submits a query → Admin responds	Playwright/Cypress
Security	Auth bypass, role escalation, SQL injection, XSS, delete/account-management restricted to admin, query-endpoint abuse	OWASP ZAP, manual pen-test checklist
Performance	Home page + notice list load time under simulated concurrent load	k6 / Lighthouse
UAT	Real Admin/Principal/Dean/HoD/Faculty walkthroughs before go-live	Scripted scenarios, sign-off checklist

9.2 Key Scenarios

- Verify no role other than admin can successfully call any DELETE route or any /api/users route, including a direct API call bypassing the UI.
- Verify Principal, Dean, HoD, and Faculty accounts can create/edit content within scope from their own dashboards but never see a delete control or an Account Manager link.
- Verify the Student Query Form appears on “/” and is completely absent from every staff dashboard route.
- Verify a query can be submitted with no login, is captured with Name/Roll No/Department/Query, and appears in the Admin's Query Inbox.
- Verify the query-submission endpoint rejects malformed/oversized input and is rate-limited against spam.
- Verify an Alert renders in red with a functioning live countdown across major browsers/devices.
- Verify middleware.ts correctly redirects an unauthenticated or wrong-role request away from a protected dashboard route.